Project in Computer Graphics and Computer Vision (2025/26)

Generating synthetic data in Unreal

Ana Oliveira
MEI, PG61510

André Miranda
MEI, PG60238

Maria Matos
MEI, PG61533

***Abstract*—This work presents the generation of a synthetic dataset using Unreal Engine and its evaluation for object detection using a YOLO-based deep learning model. A real-world pen was selected as the target object and reconstructed as a 3D model. Synthetic images were generated by placing the object in simulated environments with different positions and conditions. A real image dataset was also created by capturing photographs of the same object in similar scenarios. Both datasets were annotated using the YOLO format and used to train and evaluate object detection models. The objective is to analyze the effectiveness of synthetic data and its capability to support deep learning models when compared with real-world data.**

***Index Terms*—Synthetic data generation, Unreal Engine, YOLO object detection, deep learning, computer vision, 3D reconstruction, object detection, dataset generation**

I. Introduction

Deep learning-based object detection systems require large amounts of annotated data to achieve reliable performance. However, collecting and manually labeling real images can be time-consuming and expensive. Synthetic data generation provides an alternative approach by allowing the creation of controlled datasets using realistic 3D environments.

In this project, Unreal Engine was used to generate synthetic images of a pen in different simulated scenarios. The generated dataset was compared with a real dataset containing photographs of the same object. Both datasets were tested using a YOLO object detector to evaluate the impact of synthetic data on detection performance.

II. Real Object and Environment Data Acquisition

The initial stage of this project consisted of collecting visual information from the real-world object and its surrounding environments. After some consideration between each other, we decided to use a unique pen as the target object for detection (Figura 1), which was chosen as the target object for the complete synthetic data generation and object detection pipeline.

A set of images of the physical pen was acquired from different perspectives and orientations in order to capture its visual characteristics, including its shape, dimensions, colors, and surface details. These images served as the main reference for the following stages, particularly for the creation of the 3D model.

Figure 1. Image of the real pen that is used throught the whole project.

In addition to the object acquisition, images of possible environments where the pen could naturally appear were also collected (Figura 2). These scenarios provided references for the creation of realistic virtual environments inside Unreal Engine, allowing the generated synthetic images to reproduce conditions closer to real-world situations.

Figure 2. Image of the real pen in a real scenario.

III. 3D Reconstruction of the Object

After collecting the reference images, the next step consisted of creating a digital representation of the pen. The objective of this stage was to reproduce the physical object as accurately as possible in a 3D environment.

The reconstruction process focused on representing the geometry and visual properties of the real object. Features such as the shape, proportions, and external appearance of the pen were considered to ensure that the generated model maintained similarities with the real object.

At first we attempted to make it from scratch on the Unreal Engine (Figura 3) but the results achieved deemed to be doable but also too time consuming.

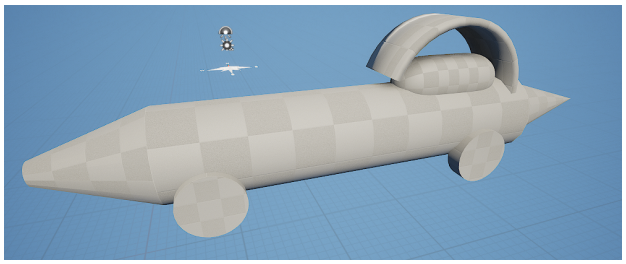

Figure 3. Prototype of the synthetic pen made in Unreal Engine.

By the end of everything, we decided to use application that let us take photos of our pen and then it generated the model in 3D instead of us trying to put the 3D model and the texture of the pen from scratch.

The resulting 3D model was then prepared for integration into Unreal Engine, where it could be placed in different environments and used for synthetic image generation (Figura 4).

Figure 4. Final 3D model of the synthetic pen.

IV. Unreal Engine Scenario Creation and Synthetic Dataset

After reconstructing the 3D model of the pen, the next stage consisted of creating the virtual environments used for synthetic data generation.

The scenarios were developed in Unreal Engine based on the previously collected images of real environments. The objective was to reproduce realistic conditions where the pen could appear, including similar backgrounds, object positioning, illumination, and camera perspectives.

The reconstructed pen model was placed inside these environments with different orientations and positions. By modifying scene parameters such as lighting, camera angle, and object placement, multiple variations of the same scenario were generated.

This approach allows the creation of a diverse synthetic dataset while maintaining complete control over the conditions in which the object appears.

Multiple virtual scenarios were created based on the previously collected environment references. Variations were introduced in camera position, object orientation, lighting conditions, background elements, and object placement.

The purpose of this stage was to create a dataset containing diverse examples of the pen under different conditions. These images simulate the variability normally present in real-world image acquisition and provide training data for the object detection model.

Figure 5. Virtual scenario created in Unreal Engine containing the reconstructed pen.

V. Real Dataset Creation

To evaluate the effectiveness of synthetic data, a real image dataset was created containing photographs of the pen in environments similar to the simulated scenarios.

The real dataset provides a reference for evaluating how well the model trained with synthetic images can generalize to real-world conditions. Unlike synthetic images, real images contain natural variations caused by camera limitations, lighting changes, shadows, and background complexity.

The comparison between these datasets allows the analysis of the potential benefits and limitations of synthetic data generation.

VI. Real Dataset Annotation

The real images acquired from the physical pen require manual annotation before being used for object detection training.

The annotation process was performed using LabelImg, a graphical annotation tool that allows the creation of bounding boxes around objects in images.

For each image, a bounding box was manually placed around the pen and assigned to the corresponding object class. The annotations were exported in YOLO format, generating a text file for each image.

Each YOLO annotation file contains the following information:

```
class_id x_center y_center width height
```

For example:

```
15 0.626333 0.527500 0.300000 0.243000
```

The values are normalized according to the image dimensions, allowing YOLO to process images with different resolutions.

Figure 6. Bounding box annotation of the pen using LabelImg.

VII. Synthetic Dataset Annotation

Unfortenatly due to the lack of time and the fact that Unreal Engine kept giving us problems, we were unable to run a script on Unreal Engine in where it would generate photos of the different scenarios and positions in where the pen could be.

The idea was to move both the pen and the camera in different scenarios, making sure that the pen is within the camera's lens, and run the script to recreate a limited number of images of such scenarios (between 100 or less images) and then switch to a different place of the map. While at the same time, the script would also generate the YOLO annotation of the image obtained in a different folder to not mix up the images with the text file.

But as said, due to the lack of time, we took a few images from the map that we had created to do the datset and used again the labelImg to help in creating the YOLO annotations (Figura 7).

VIII. YOLO Object Detection Training

The object detection models were implemented using the YOLOv8 architecture provided by the Ultralytics library. This framework was selected due to its balance between accuracy and computational efficiency, making it suitable for experiments involving both small and synthetic datasets.

Figure 7. Bounding box annotation of the pen using LabelImg for the synthetic dataset.

All experiments were initialized using pre-trained weights (yolov8n.pt), enabling transfer learning from large-scale datasets and improving convergence speed during training.

The training pipeline was executed in Python as follows:

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")

model.train(
    data="dataset.yaml",
    epochs=50,
    imgsz=416,
    batch=4
)
```

Three different experimental setups were defined to evaluate the influence of synthetic data:

- **Real dataset training:** The model was trained exclusively on manually annotated real images of the pen.
- **Synthetic dataset training:** The model was trained using images generated in Unreal Engine, containing controlled variations in lighting, camera position, and background.
- **Combined dataset training:** Both real and synthetic images were merged to evaluate whether data augmentation

improves generalization performance.

Each configuration was trained under identical hyperparameters to ensure a fair comparison between datasets. The training process optimized bounding box regression, classification loss, and objectness confidence simultaneously.

The goal of this stage was to evaluate whether synthetic data alone can support robust model training and whether its combination with real data improves overall detection performance.

IX. Model Evaluation

After training, the performance of each YOLO model was evaluated using the built-in validation framework of Ultralytics YOLOv8. The evaluation was performed on unseen test images to measure the generalization capability of each model.

The evaluation process was implemented as follows:

```
metrics = model.val(data="dataset.yaml")
print(metrics)
```

The following standard object detection metrics were considered:

- **Precision:** Measures the ratio of correct positive detections to total predicted positives.
- **Recall:** Measures the proportion of ground truth objects correctly detected by the model.
- **mAP@0.5:** Mean Average Precision using an IoU threshold of 0.5, representing standard detection accuracy.
- **mAP@0.5-0.95:** A stricter metric that evaluates performance across multiple IoU thresholds, providing a more complete assessment of detection quality.

In addition to standard evaluation, inference on test images was performed to visually inspect detection performance:

```
model.predict(
    source="dataset/images/test",
    save=True,
    conf=0.25
)
```

The predictions were saved with bounding boxes drawn on the detected objects, allowing qualitative analysis of detection accuracy.

A cross-domain evaluation was also conducted, where a model trained on synthetic data was tested on real images. This experiment is particularly important as it highlights the domain gap between simulated and real-world data distributions.

Finally, all models were compared under identical evaluation conditions in order to determine the impact of dataset composition on object detection performance.

X. Results

The results obtained from the experiments show the differences in performance between models trained on real, synthetic, and combined datasets.

The real-trained model achieved strong performance on real test images, as expected due to the direct correspondence between training and testing data. However, its performance decreased when evaluated on synthetic data due to domain differences.

The synthetic-trained model demonstrated lower performance on real images, indicating a domain gap between simulated and real environments. However, it still managed to detect the object in most cases, showing that synthetic data provides useful learning signals.

Finally, the combined dataset approach achieved the best overall performance, benefiting from both the realism of real data and the variability of synthetic data. This suggests that synthetic data can effectively complement real data when properly integrated.

Table I
YOLO performance comparison

Training Data	Precision	Recall	mAP@0.5
Real			0.995
Synthetic			
Real + Synthetic			

XI. Conclusion

This project presented a complete workflow for the generation of synthetic data and its evaluation for object detection using a YOLO-based deep learning model. Starting from the acquisition of real images of a pen and its surrounding environments, a 3D representation of the object was created and integrated into Unreal Engine to generate realistic simulated scenarios.

The synthetic dataset produced in Unreal Engine provided controlled variations in object position, camera perspective, illumination, and environment conditions. In parallel, a real dataset was collected and manually annotated using LabelImg, producing YOLO-compatible bounding box annotations.

Both datasets were prepared and evaluated using YOLO object detection models in order to compare the impact of synthetic and real data. CONTINUAR QUANDO TIVER OS RESULTADOS OBTIDOS

References

[1] PyTorch Documentation, https://pytorch.org/
[2] G. Jocher et al., "YOLO by Ultralytics", 2023.
[3] Epic Games, "Unreal Engine Documentation", https://www.unrealengine.com